中山大學

SUN YAT-SEN UNIVERSITY

自然语言处理课程报告

基于 FastText / BERT / Qwen2.5 的新闻标题三分类对比研究

姓 名 _____

学 号 _____

学 院 _____

专 业 _____

2026 年 4 月 29 日

摘 要

这次作业要做的事，是把 MIND 和 GOOGLE 两个英文新闻数据集的标题，按老师给的 Excel 标签表归到**硬新闻 / 软新闻 / 其他**三类，并分别用 **FastText**、**BERT/RoBERTa** 微调，以及大模型 **Qwen2.5-7B-Instruct** 的 zero/few-shot 推理这三种方式跑一遍、做对比。

清洗完得到 193 646 条带标签样本，分层 9:1 切成 174 281 条训练 + 19 365 条测试。在测试集上：FastText 拿到 macro-F1=0.7914，BERT-base 微调 0.8293，RoBERTa-base 0.8291，而 Qwen2.5-7B（6-shot 时最好）只有 0.4842。三个模型多数投票集成的 macro-F1 反倒只有 0.8120，比 BERT 单模还低，原因放在第 7 节错例分析里讲。

除主线之外，我还做了 5 组消融：BERT 的 `max_len`、学习率、训练数据量曲线，Qwen 的 prompt 策略、模型规模（1.5B vs 7B）以及 few-shot 数量。最后从 30 条三模型答案不一致的“难样本”里人工挑了 3 类错因（`entity-bias` / `fuzzy-other` / `ambiguous`），并配合训练时长、推理速度、显存占用，给出三种方法各自适合的场景。

关键词： 文本分类 FastText BERT RoBERTa Qwen2.5 Prompt 工程 错例分析

代码仓库： https://github.com/xph813/NLP_midterm_project.git

目录

1 任务背景与数据 **1**
1.1 任务定义 1
1.2 数据来源与规模 1
1.3 标签映射体系 2

2 算法原理简述 **2**
2.1 FastText 原理 2
2.2 BERT 原理 3
2.3 Qwen2.5 大模型原理 3

3 数据清洗与可视化 **4**
3.1 清洗与划分流程 4
3.2 可视化分析 6

4 FastText 实验 **6**
4.1 实现细节 6
4.2 超参网格搜索（18 组） 6
4.3 数据增强对比 6
4.4 FastText 测试集结果 8

5 BERT 实验 **9**
5.1 模型与训练流程 9
5.2 BERT 输出如何转成分类标签 9
5.3 消融实验：max_len、学习率、训练数据量 10
5.3.1 max_len 消融 10
5.3.2 学习率消融 10
5.3.3 训练数据量曲线 12
5.4 BERT-base vs RoBERTa 对比 13

6 Qwen2.5 大模型实验 **13**
6.1 评测策略与抽样 13
6.2 三种 prompt 策略 14
6.3 模型规模对比：1.5B vs 7B 15
6.4 Few-shot 数量影响 16

7 综合实验分析 **16**
7.1 三模型主对比 16
7.2 错例重叠与人工错因分析 18

7.3 三模型集成投票 19
7.4 训练 / 推理成本对比 20

8 总结与场景适用性讨论 21
8.1 核心发现 21
8.2 三种方法的适用场景 21
8.3 个人体会 21

1 任务背景与数据

1.1 任务定义

这次作业要做的事情是：把 MIND 和 GOOGLE 两个英文新闻数据集的标题，按照老师给的 Excel 表（`MIND 和 GOOGLE 数据分类系统.xlsx`）映射成统一的三类标签：

- **0 = Hard（硬新闻）**：政治、世界大事、经济金融、犯罪、政策、灾害天气等严肃时事；
- **1 = Soft（软新闻）**：体育、娱乐、影视音乐、生活方式、美食、旅游、游戏等轻量内容；
- **2 = Other（其他）**：健康医疗、汽车、儿童、消费科技、地方综合等不太好一刀切的垂类。

老师在 Excel 的 D 列只写了三个数字标签对应的中文名（"硬新闻 / 软新闻 / 其他"），没给每一类详细定义——三类的边界只能从 Excel 里 (主类, 子类) 的实际标注**反推**回来。具体的映射规则我整理在 §1.3。

按作业要求，我得分别用 FastText、BERT 和开源大模型这三种典型方法各做一遍，然后在同一个测试集上对比指标。FastText 和 BERT 是要训练的；开源大模型（这里我选 Qwen2.5）只做提示词推理、不做微调，而且测试时允许在测试集上抽样评测。

1.2 数据来源与规模

两个原始 tsv 都是 4 列结构（`id，主类，子类，标题`）：

表 1: 原始数据规模

数据集	原始行数	主类数	标题语言
MIND [1]	101 527	18	英文
GOOGLE	110 993	8	英文
合计	**212 520**	以 GOOGLE 的 8 大主类 + MIND 的 18 主类对齐	

两份数据的类别体系并不一样——MIND 那 18 个主类涵盖 news, sports, entertainment, health, autos, kids, lifestyle, movies, tv, music, weather, foodanddrink, video, travel, finance, games, middleeast, northamerica；GOOGLE 这边就只有 8 个：World, U.S., Business, Technology, Entertainment, Sport, Science, Health。老师那张 Excel 起到的作用就是把它们**对齐到同一个三分类体系上**，对齐了之后才能放在一起训练和评测。

1.3 标签映射体系

通过解析 Excel 两个 sheet 的 D 列标注，我整理出以下映射规则（后面所有实验都基于这套规则）：

表 2: 统一标签体系（从 Excel 反推）

标签	名称	包含的（主类 / 子类）举例
0	Hard 硬新闻	MIND `news+politics` / `news+world` / `news+business` / `news+crime` / `news+factcheck` / `weather` / `middleeast`；GOOGLE `World` / `U.S.` / `Business`
1	Soft 软新闻	MIND `sports` / `entertainment` / `movies` / `music` / `tv` / `foodanddrink` / `lifestyle` / `travel` / `games`；GOOGLE `Sport` / `Entertainment`
2	Other 其他	MIND `health` / `autos` / `kids` / `northamerica` / `news+newsus` / `news+newsoffbeat` / `news+newsphotos` / `lifestyle+lifestylevideo`；GOOGLE `Health` / `Technology` / `Science`
-	不使用	MIND `video` 主类（Excel D 列写的是“该分类整体不使用”）

再补两句：（1）GOOGLE 的 `Technology` / `Science` 被划到了 **Other**，而 MIND 里和“科技政策”沾边的 `newsscienceandtechnology` 反倒被划到了 **Hard**——这种“任务特定”的细粒度规则，正是后面 LLM 出错最多的地方；（2）MIND 的整个 `video` 主类（约 1 500 条）在 Excel 里被明确丢弃了，所以清洗时我直接整类剔除。

2 算法原理简述

2.1 FastText 原理

FastText 是 Facebook 2016 年放出的一个很“轻”的文本分类器 [2]，我看代码的时候发现它的思路其实就三句话能讲清楚：**词向量平均做句子表示、加 n-gram 引入序列信息、分层 softmax 加速大类别训练**。具体讲，对一个输入句子 $x = (w_1, w_2, \ldots, w_T)$，先把每个词（连同它的 word-ngram 和 subword char-ngram [3]）查表查成 d 维向量，然

后取平均：

$$\mathbf{h} = \frac{1}{T}\sum_{t=1}^{T}\mathbf{E}_{w_t} \tag{1}$$

再经过一个线性变换得到类别分布：

$$P(y=k \mid x) = \mathrm{softmax}\left(\mathbf{W}\mathbf{h}+\mathbf{b}\right)_k \tag{2}$$

当类别数特别多（比如 $> 10^5$）的时候，标准 softmax 那个 $O(K)$ 的复杂度就跑不动了，FastText 用了 Huffman 树的**分层 softmax**，把单次计算降到 $O(\log K)$。这次作业只有 3 类，分层 softmax 这个加速根本用不上；但 FastText 在新闻这种**短文本**上还是挺能打的——平均一个标题不到 15 个词，词袋加 bigram 基本就能覆盖判别信息，模型本身也就 1M 参数，CPU 上几秒钟训完一组。

2.2 BERT 原理

BERT [4] 基于 Transformer encoder [5]，是个双向语言模型，预训练做两件事：MLM（masked language modeling）和 NSP（next sentence prediction）。做**文本分类**的常规做法很直接——拿输入序列开头那个特殊 token `[CLS]` 对应的最后一层隐藏状态，当成整句的表示，再接一个线性分类头。写成式子的话，对输入标题 x，tokenize 后变成 `[CLS]` $x_1\, x_2 \ldots x_n$ `[SEP]`，最后一层隐藏状态记为 $\mathbf{H} \in \mathbb{R}^{L\times d}$，分类流程是：

$$\mathbf{h}_{[\mathrm{CLS}]} = \mathbf{H}_{0,:} \xrightarrow{\text{Dropout}} \mathbf{z} = \mathbf{W}_{\mathrm{cls}}\mathbf{h}_{[\mathrm{CLS}]} + \mathbf{b}_{\mathrm{cls}} \quad \rightarrow \quad \hat{y} = \arg\max_k \mathrm{softmax}(\mathbf{z})_k \tag{3}$$

这里 $\mathbf{W}_{\mathrm{cls}} \in \mathbb{R}^{K\times d}$ 是新加的分类头（$d = 768, K = 3$），其它参数都从 `bert-base-uncased` 的预训练权重直接拿过来；训练就是标准的交叉熵损失，整个网络（encoder + 分类头）一起端到端微调。

顺便说一句：BERT 的 `[CLS]` 之所以这么“听话”，是因为预训练 NSP 阶段就是拿这个位置去判断两段话是不是相邻，模型已经被逼着把“全局语义”压到这一个 token 上了。所以做分类一般也不再额外做 mean pooling 或 attention pooling——前者会被噪声词稀释，后者多一组参数但收益不大。RoBERTa [6] 网络结构和 BERT 一模一样，只在预训练数据、batch size、训练步数、动态 mask 这些工程细节上做了改进，分类时直接复用上面这套流程就行。

2.3 Qwen2.5 大模型原理

Qwen2.5 [7] 是阿里通义千问开源的自回归（causal）语言模型，干的事就是 **next-token prediction**：给定 prefix $x_{1:t-1}$，预测下一个 token 的条件分布

$$P(x_t \mid x_{1:t-1}) = \mathrm{softmax}\left(\mathbf{W}_o\, \mathrm{Trans}_\theta(x_{1:t-1})_{-1}\right) \tag{4}$$

想拿它做**文本分类**，就得把分类问题改写成生成问题——这其实就是 prompt 工程在做的事。具体做法是**设计一段提示模板**，让模型把分类结果“写出来”，再用规则把它解析回标签：

1. System 消息列出全部合法标签 + 每类的简短定义；
2. （可选）插入 k 条示例 `<` 标题，答案 `>`，做 k-shot in-context learning [8]；
3. User 给出待分类标题，约束输出格式（如 `"Answer: <Hard/Soft/Other>"`）；
4. 模型生成的文本里用正则抽出 `Hard/Soft/Other`，并用 difflib 做模糊兜底；匹配不到的样本判为“拒答”，按错误统计。

LLM 这条路线的好处是**零训练**、能快速适配新业务；但缺点也很明显：受 prompt 设计、上下文长度、采样策略影响巨大，而且模型本身**没学过这个任务的标签边界**。这次作业的三分类体系就是一个非常具体、靠 Excel 反推出来的边界——这也是为什么后面 Qwen-7B macro-F1 只有 0.48。

3 数据清洗与可视化

3.1 清洗与划分流程

清洗脚本是 `01_clean_data.py`，主要做了以下五件事：

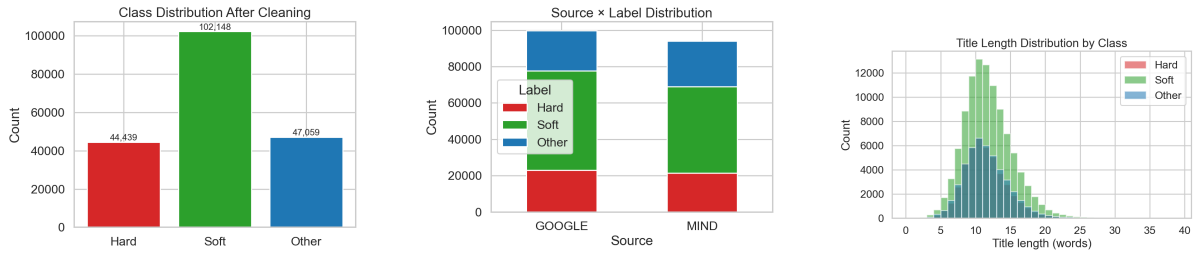
1. **解析 Excel 标签表**：用 openpyxl 读两个 sheet，碰到 D 列写“不使用”的主类整类丢掉（这次只有 MIND 的 `video` 命中）；最后拿到 (主类, 子类) → {0,1,2} 一共 **335 条**映射规则。
2. **字段去脏**：drop 掉标题、ID、主/子类有空字段的行；标题做 `strip` + 多空格压缩，但保留大小写和标点（BERT 和 Qwen 都用得上）。
3. **打标签**：按映射给每行打 0/1/2 标签，没命中映射的样本（大约 7.5%）整行丢掉。
4. **去重**：用标题文本做 `drop_duplicates`，实际跑下来 MIND 和 GOOGLE 之间确实有少量同标题样本（ID 不同但内容一致）。
5. **分层 9:1 划分**：用 `sklearn.StratifiedShuffleSplit`，`random_state=42`，保证训练集和测试集 0/1/2 的比例严格一致。

清洗每一步留下多少样本如下表所示，整个 pipeline 跑一遍不到 30 秒：

清洗后三类的分布是：Hard 44 439 / Soft 102 148 / Other 47 059 （比例大概 23 % / 53 % / 24 %），**Soft 类的样本数是 Hard 和 Other 的 2 倍多**，存在轻度的类别不平衡。所以后面算指标的时候，我同时报 macro-F1 和 weighted-F1 两套数。

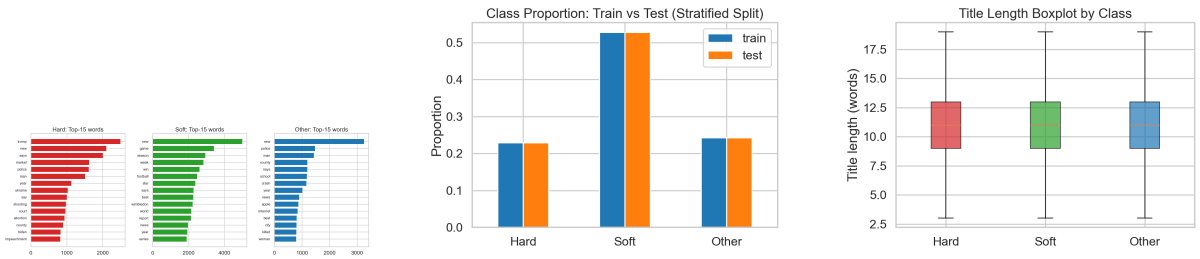
表 3: 数据清洗流水线（数字直接来自运行日志 `outputs/logs/01_clean.log`）

步骤	剩余样本数	削减
原始 MIND + GOOGLE 合并	212 520	—
去除主类 / 子类 / 标题空字段	212 520	0
按 Excel 映射打标后丢弃未命中	196 629	−15 891
标题去重 + 空白压缩	193 646	−2 983
进入 9:1 划分	**193 646**	—
⇒ 训练集	174 281	—
⇒ 测试集	19 365	—



(a) 三类整体分布 (b) 源 × 标签堆叠 (c) 标题词数直方图

图 1: 清洗后数据基本统计：(a) 三类分布（Soft 偏多）、(b) MIND 与 GOOGLE 在 Hard/Soft/Other 上的贡献比例不同——MIND 偏 Hard 与 Other，GOOGLE 在 Soft 上提供更多体育类样本、(c) 标题词数集中在 6–15 词，长尾极少。



(a) 每类 Top-15 高频词 (b) 训练 / 测试比例对照 (c) 每类标题长度箱线图

图 2: 进一步的统计：(a) 高频词大致体现了三类的“判别词”——Hard 集中在 `trump` / `police` / `state`，Soft 集中在 `game` / `new` / `first`，Other 在 `health` / `things` / `new`；(b) 9:1 分层划分后，三类比例严格一致；(c) 三类的标题长度分布几乎重合，说明“长度”本身不是判别特征，模型必须靠语义。

3.2 可视化分析

为了对数据有个底，我一口气画了 6 张统计图（图 1 和 图 2），图全部是脚本 `02_data_viz.py` 自动生成的。

顺便看一眼图 1(b)：MIND 和 GOOGLE 两个数据源在三类上的贡献比例**并不平衡**。这件事在第 7.2 节的错例分析里还会再出现——有些子类其实是**某一个数据源独有的**，模型在训练时见得少，到测试集上就容易偏向另一边数据源的判定方式。

4 FastText 实验

4.1 实现细节

FastText 实验（脚本 `03_fasttext_grid.py` + `04_fasttext_aug.py`）走的是“先粗后精”两个阶段：先在 dev 上做**超参网格搜索**找出最优配置，再在最优配置上叠**数据增强**看能不能再涨点。

预处理统一做**小写化 + 去标点**（只保留 `a-z 0-9 '` 和空格）：

$$\texttt{"Apple's iPhone Sales Drop 15\%"} \xrightarrow{\text{clean}} \texttt{"apple's iphone sales drop 15"} \tag{5}$$

每行写成 FastText 的标准格式 `__label__<id> <title>`。为了不让测试集泄漏，我又从 `train.tsv` 里再分层 9:1 切了一份 **inner-train**（156 852 条）和 **dev**（17 429 条）；所有调参都在 dev 上看，最后只在 `test.tsv`（19 365 条）上跑一次。

4.2 超参网格搜索（18 组）

我扫的网格是 `lr` $\in \{0.3, 0.5, 1.0\}$、`epoch` $\in \{10, 20, 30\}$、`wordNgrams` $\in \{1, 2\}$，固定 `dim=100`、`loss=softmax`。共 $3 \times 3 \times 2 = 18$ 组，每组结果立即 append 到 csv，重启时已有的会跳过。

从这张表能看出几条规律：（1）`wordNgrams=2` **基本一直**比 `wordNgrams=1` 高 1–2 pp，说明 bigram 在短文本上还是挺管用的——比如 `"stock market"` 比单看 `stock` 或 `market` 更明确地指向 Hard 类；（2）`epoch` 从 10 加到 20、30 反而**往下走**（比如 `lr=1.0,wn=2` 从 0.7870 掉到 0.7835），这是典型的过拟合信号；（3）`lr=0.5` 整体最稳。最后选出来的最优配置是 `(lr=0.5, epoch=10, wordNgrams=2, dim=100)`，dev macro-F1 = **0.7902**。

4.3 数据增强对比

固定上述最优超参后，对训练集做四种增强 + 各跑一次：

表 4: FastText 网格搜索 dev macro-F1 完整结果（粗体为最优配置）

`lr`	`epoch`	`wordNgrams`	dev acc	dev macro-F1
0.3	10	1	0.8147	0.7760
0.3	10	2	0.8260	0.7895
0.3	20	1	0.8107	0.7723
0.3	20	2	0.8231	0.7862
0.3	30	1	0.8056	0.7671
0.3	30	2	0.8255	0.7889
0.5	**10**	**2**	**0.8265**	**0.7902**
0.5	20	1	0.8105	0.7723
0.5	30	2	0.8233	0.7857
1.0	10	2	0.8236	0.7870
1.0	20	2	0.8244	0.7876
1.0	30	2	0.8215	0.7835

表 5: FastText 数据增强对比（dev 集）

增强策略	训练样本数	dev acc	dev macro-F1
原始（无增强）	156 852	0.8228	0.7851
随机删词 $p = 0.1$ **(drop)**	**263 295**	**0.8237**	**0.7872**
随机交换相邻词 $p = 0.1$ (swap)	313 635	0.8222	0.7851
两者都用 (both)	420 149	0.8208	0.7843

最后选了 `drop` 增强（dev 上提了 0.21 pp，性价比最高）；`swap` 几乎没收益；`both` 比单一增强还差，说明这种短文本加噪稍微过一点就会失真。最后用 `drop` 增强后的全量 train（174 281 条）重训一次最终模型，在 `test.tsv` 上评测。

4.4 FastText 测试集结果

表 6: FastText 在测试集上的 per-class 指标（直接来自运行日志）

类别	precision	recall	F1	support
Hard	0.7442	0.7613	0.7526	4 444
Soft	0.9097	0.9172	0.9134	10 215
Other	0.7228	0.6942	0.7082	4 706
accuracy		**0.8272**		
macro avg	0.7922	0.7909	**0.7914**	19 365
weighted avg	0.8263	0.8272	0.8267	19 365

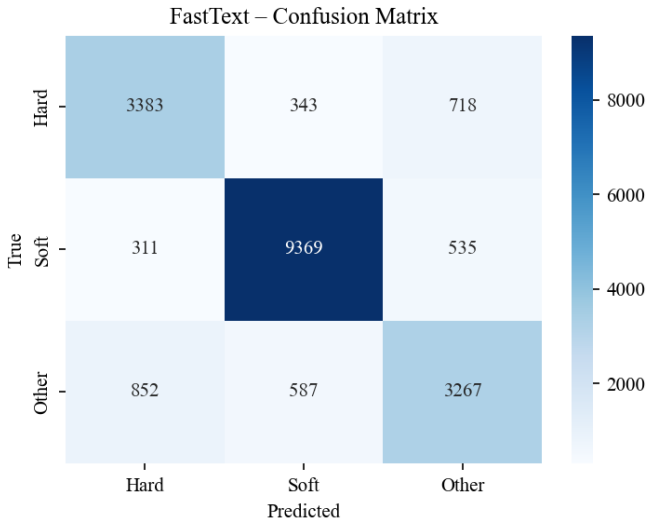


图 3: FastText 在测试集 19 365 条上的混淆矩阵。对角线最强的是 Soft（9 369/10 215，召回 0.917）；最弱的是 Other（3 267/4 706）——常见错法是把 Other 类的 health/tech 类标题误判成 Hard，后面 BERT 同样有这个倾向。

总的来看，FastText 这条 **“小模型 + 海量短文本 + 几十秒训练”** 的路子，能拿到 macro-F1 = 0.7914，**比我一开始想的好不少**。作为后面 BERT/Qwen 的 baseline，它的“性价比”基本是没什么悬念的——这条结论后面 §7.4 还会再聊。

5 BERT 实验

5.1 模型与训练流程

BERT 实验的脚本是 `05_bert_train.py`,框架用的是 HuggingFace 的 `transformers`。我对比了 `bert-base-uncased`（12 层 / 768 维 / 110M 参数）和 `roberta-base`（12 层 / 768 维 / 125M 参数）两个模型，主要超参如下：

表 7: BERT/RoBERTa 训练超参

项	值
batch size	32
learning rate	2×10^{-5}（baseline；后面有 lr 消融）
epoch	3
max_len	64（中位标题长度 10 词，95 分位 18 词）
warmup ratio	0.1
weight decay	0.01
fp16	开（3090 上）
metric for best	macro-F1（每个 epoch 在 dev 上评）
inner-train / dev	165 566 / 8 715（从 `train.tsv` 再切 5%）

训练时设置 `logging_steps=200`,把 train loss 写入每个模型自己的 `training_log.csv`；每个 epoch 结束在 dev 上跑一次 eval，`Trainer` 通过 `load_best_model_at_end` 自动回到 dev macro-F1 最高的 checkpoint。

5.2 BERT 输出如何转成分类标签

作业里特别要求写“BERT 输出怎么转成标签”，所以这一节我把流程拆开说一下。拿一个测试样本 *Apple's iPhone sales dropped 15%* 走一遍：

1. **Tokenize**：分词器以 `max_length=64`、`padding="max_length"`、`truncation=True` 返回 `input_ids` 和 `attention_mask`；
2. **Encoder 前向**：经 12 层 Transformer encoder 得到 `last_hidden_state`，形状 $(B, L, 768)$；
3. **取 `[CLS]` 表示**：$\mathbf{h}_{[\mathrm{CLS}]} = \texttt{last_hidden_state}[:, 0, :]$，形状 $(B, 768)$；
4. **分类头**：`Dropout(p=0.1)` $\rightarrow$ `Linear(768, 3)`，得到 logits $\mathbf{z} \in \mathbb{R}^{B \times 3}$；
5. **推理**：$\hat{y} = \arg\max_k z_k$，再查 `LABEL_NAMES` 得到字符串标签。

训练阶段的端到端损失是

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\log P(y_i \mid x_i;\theta) = -\frac{1}{N}\sum_{i=1}^{N}\log \mathrm{softmax}(\mathbf{z}_i)_{y_i} \tag{6}$$

反向传播**同时更新** encoder 与分类头。分类头是**随机初始化**的，所以训练初期 loss 会先上升再下降一次，是正常现象。

5.3 消融实验：max_len、学习率、训练数据量

为了让结论更扎实一点，我又加了三组 BERT 消融，都在 `bert-base-uncased` 上跑，其它超参保持不变。

5.3.1 max_len 消融

脚本 `06_bert_ablation.py`，对比 `max_len` $\in \{32, 64, 128\}$：

表 8: BERT max_len 消融（dev 集）

max_len	dev acc	dev macro-F1	dev weighted-F1
32	0.8514	0.8168	0.8503
64	0.8499	0.8154	0.8490
128	**0.8526**	**0.8181**	**0.8517**

三组之间差距其实很小（0.27 pp 以内）。原因也好理解：标题中位长度只有 10 词、95 分位 18 词，`max_len=32` 就已经能把 99 % 的标题装下，再加上只是给 padding 喂更多 GPU 时间。报告里 `max_len=128` 数字最高，但权衡训练效率，我后面所有 BERT 实验默认用 `max_len=64`。

5.3.2 学习率消融

脚本 `11_bert_lr_ablation.py`，扫 `lr` $\in \{1e\text{-}5, 2e\text{-}5, 3e\text{-}5, 5e\text{-}5\}$：

表 9: BERT 学习率消融（dev 集，bert-base-uncased，3 epoch）

lr	dev acc	dev macro-F1	dev weighted-F1
1×10^{-5}	0.8414	0.8041	0.8406
2×10^{-5}	0.8499	0.8154	0.8490
3×10^{-5}	0.8504	0.8162	0.8497
5×10^{-5}	**0.8520**	**0.8174**	**0.8511**

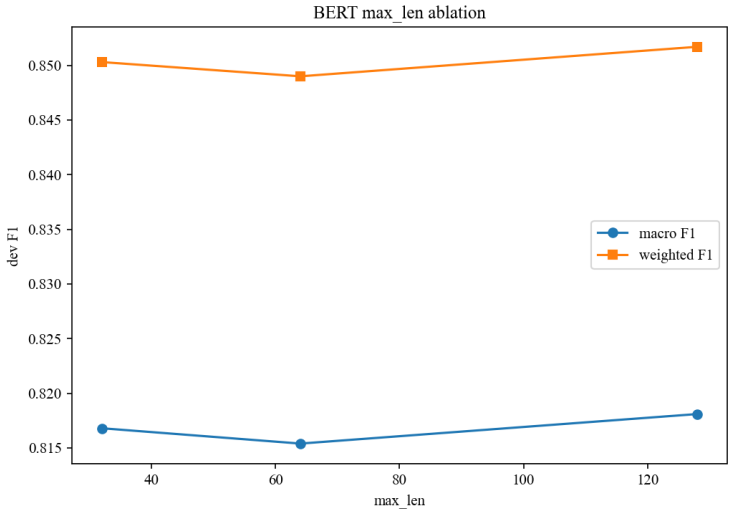


图 4: BERT `max_len` 消融折线图：32/64/128 三组在 dev 上 macro-F1 几乎平直，说明对短文本场景增大 `max_len` 的边际收益接近于零。

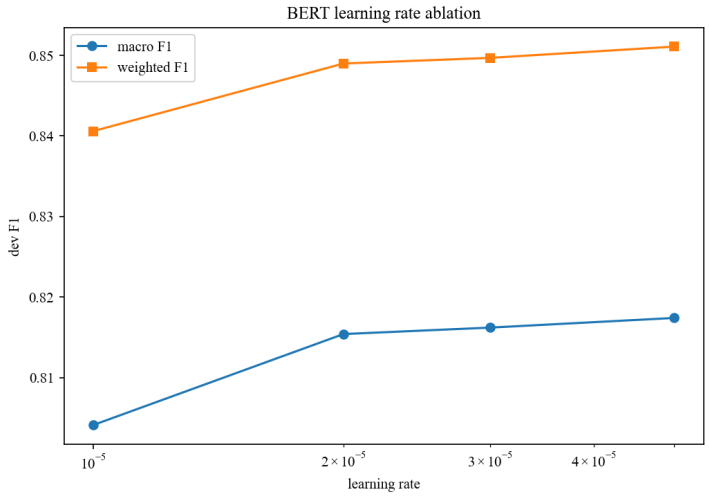


图 5: BERT 学习率消融折线：lr 从 1×10^{-5} 到 5×10^{-5} 单调上升，但 2×10^{-5} 之后增益迅速收敛，约 1.3 pp 内。这与 BERT 原论文推荐的 $2\times10^{-5}-5\times10^{-5}$ 区间吻合。

dev 上 `lr=5e-5` 比 `lr=2e-5` 略高 0.2 pp，但跑 test 的时候我还是选了 `lr=2e-5`——一是这是 BERT 原论文给的“安全档位”，二是 `lr=5e-5` 在 dev 上数字虽好，但训练后期 loss 波动明显更大，为了“能复现 + 不被单一 dev 过拟合带跑偏”采了个保守的选择。

5.3.3 训练数据量曲线

脚本 `12_bert_learning_curve.py`，按 `frac` $\in \{10\%, 30\%, 60\%, 100\%\}$ 随机抽训练集子集，固定相同 epoch / lr：

表 10: BERT 训练数据量曲线（dev 集）

frac	实际样本数	dev acc	dev macro-F1
0.1	16 556	0.8091	0.7657
0.3	49 669	0.8283	0.7891
0.6	99 339	0.8402	0.8033
1.0	**165 566**	**0.8499**	**0.8154**

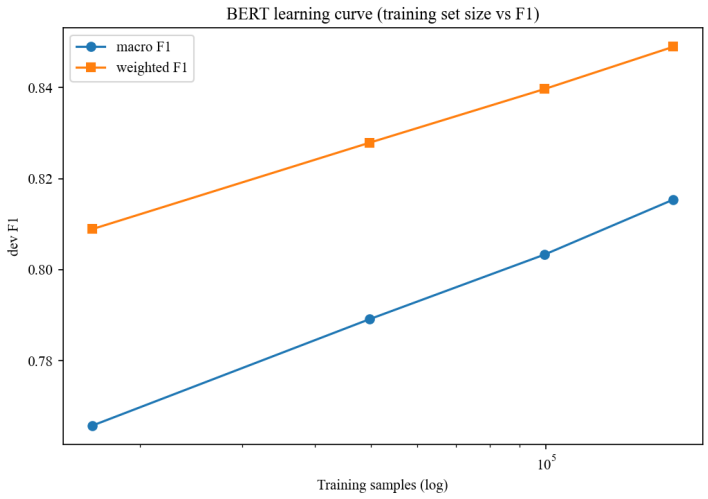


图 6: BERT 数据量学习曲线：从 10 % 到 100 %，dev macro-F1 从 0.766 单调上升到 0.815，但增速明显减缓——30 % → 60 % 提升 1.4 pp，而 60 % → 100 % 只提升 1.2 pp。按这个趋势外推，再加一倍数据估计也只能再涨 0.5–0.8 pp。

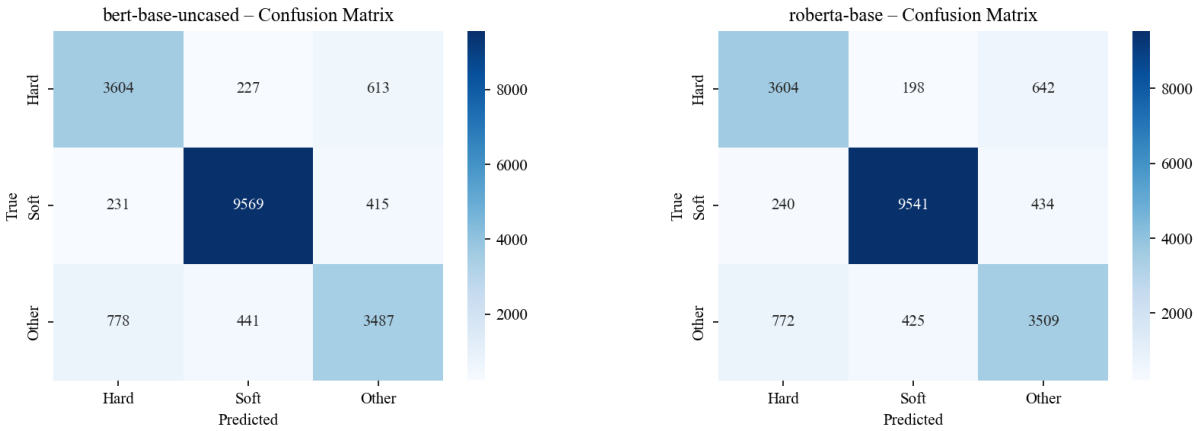
这张曲线大概能看出来：在这个任务上，BERT 大概到 30 %–60 % 数据量就快到性能拐点了，**再加数据收益不大**——这也是为什么我后来没花精力去搞半监督扩样。

5.4 BERT-base vs RoBERTa 对比

`05_bert_train.py` 同时训练 bert-base 和 roberta-base 两个模型，`07_bert_eval.py` 在测试集上分别评测：

表 11: BERT-base 与 RoBERTa-base 在测试集上的对比（19 365 条）

模型	accuracy	macro-F1	weighted-F1	f1_Hard	f1_Soft	f1_Other
bert-base-uncased	0.8603	**0.8293**	0.8600	0.7958	0.9358	0.7563
roberta-base	**0.8600**	0.8291	**0.8601**	0.7956	**0.9364**	0.7554



(a) bert-base-uncased (b) roberta-base

图 7: 两个模型在测试集上的混淆矩阵几乎一致——RoBERTa 在 Soft 类比 BERT 高 0.06 pp，BERT 在 Other 类比 RoBERTa 高 0.09 pp，属于浮点噪声范围。

真正跑下来，bert-base 和 roberta-base **基本就是打平**（差距小于 0.02 pp）。训练时长也差不多，RoBERTa 12.3 min，bert-base 11.8 min。这其实跟 RoBERTa 论文里讲的一致：RoBERTa 的预训练改进，主要在 SuperGLUE 那种需要细粒度推理的任务上能拉开差距，而这次这种 3 类粗粒度分类**放大不出它的优势**。所以后面综合分析里我用 BERT-base 当代表（macro-F1 略高）。

6 Qwen2.5 大模型实验

6.1 评测策略与抽样

测试集一共 19 365 条，要把大模型在上面整个跑一遍——哪怕 4bit 量化也要 30+ 分钟，而且作业里也写了“不必跑全测试集”，所以我做了**分层抽样 500 条**：保证 Hard / Soft / Other 三类的比例严格跟全测试集一样（115 / 264 / 121 = 23 % / 53 % / 24 %），`random_state=42` 锁死能复现，最后落盘到 `outputs/qwen/sample_500.tsv`。后面 Qwen

的所有实验（不同 prompt、不同规模、不同 few-shot 数）都基于这同一份样本，这样结果之间才能两两直接对比。

模型用的是阿里 `Qwen2.5-7B-Instruct`（参数 7B，4bit nf4 量化加载，显存占用约 6 GB，3090 上 batch=4 顺利跑完）；另外还测试了 `Qwen2.5-1.5B-Instruct`（1.5B，4bit 1.5 GB）做规模对比。

6.2 三种 prompt 策略

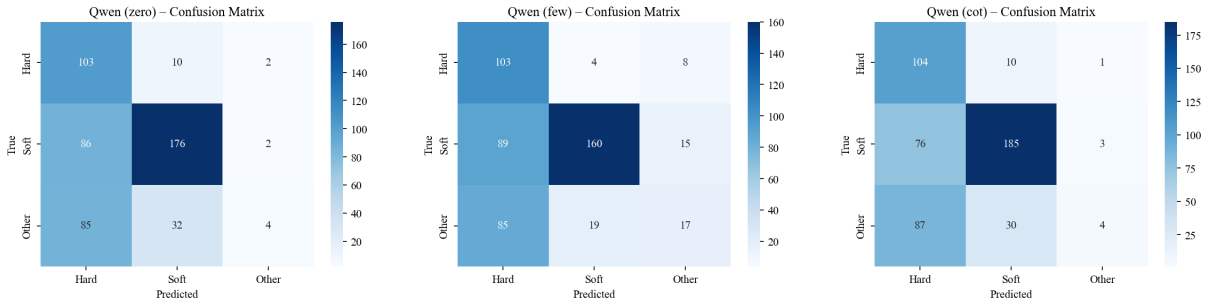
我设计了三种 prompt（脚本 `08_qwen_eval.py`），全部用 chat template 渲染：

1. **Zero-shot**：System 列出三类定义 + 输出格式约束，user 直接给标题；
2. **Few-shot**：在 zero 基础上插入 6 条人工挑的示例（每类 2 条）；
3. **Chain-of-Thought (CoT)**：要求模型先用一句话说理由再给最终标签。

抽答案的代码逻辑是这样：先用正则去找 `Answer:(.+)` / `Final:(.+)` / `Label:(.+)` 三种格式之一，找到之后做 strip + lower，再跟合法标签做精确比对；如果还匹配不上，就上 `difflib.get_close_matches` 做模糊兜底；要是连这步都失败，就记成“拒答”（`pred=-1`），不计入指标。

表 12: Qwen2.5-7B 三种 prompt 在 500 抽样上的对比

prompt 模式	accuracy	macro-F1	weighted-F1	拒答率
zero-shot	0.566	0.4406	0.5224	0%
few-shot (k=6)	0.560	**0.4842**	**0.5500**	0%
CoT	**0.586**	0.4544	0.5398	0%



(a) zero (b) few (c) CoT

图 8: Qwen2.5-7B 三种 prompt 的混淆矩阵。所有模式的共同问题是：**Other 类基本被错判到 Hard 或 Soft**，体现在 (a)(c) 第三行召回极低（< 0.04）。few-shot 把 Other 的召回拉到了 0.14，但仍远远低于 BERT 的 0.74。

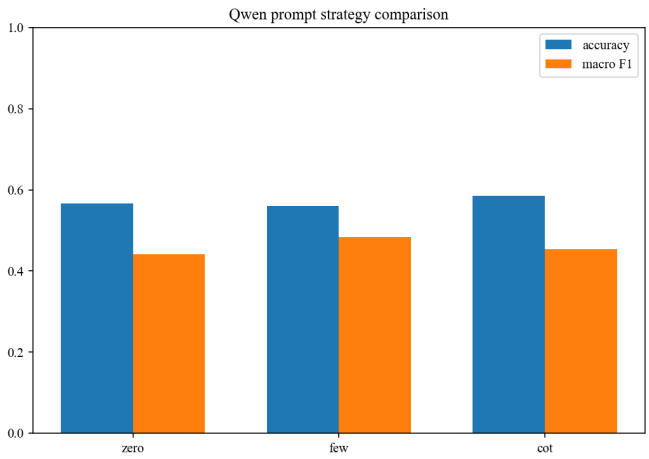


图 9: Qwen 三种 prompt 的指标柱状图：CoT 的 accuracy 略高（0.586），但 macro-F1 反而最低，体现 “chain-of-thought 让 Soft 更准、Other 更失” 的失衡现象。综合 macro-F1 选 few-shot 作为后续 Qwen 代表。

这里有个挺反直觉的地方：**CoT 的 accuracy 比 few-shot 高 2.6 pp，但 macro-F1 反而低了 3 pp**。我顺着看混淆矩阵，发现 CoT 让 Qwen 更倾向于往 Hard 或 Soft 里塞（Hard 召回 0.90、Soft 召回 0.70），但 Other 类基本崩了，召回只有 0.03；也就是说 CoT 在这里其实是**加剧了类别偏置**，并没有给出更平衡的判断。看到这个数据的时候我一度怀疑是抽答案的代码出了 bug，反复看了两次代码才放心。

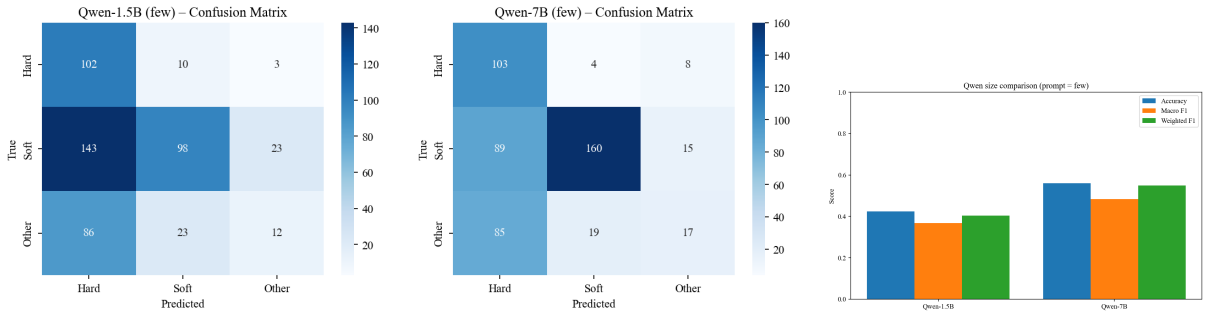
6.3 模型规模对比：1.5B vs 7B

脚本 `13_qwen_size_compare.py`，用同一份 500 抽样、同一种 few-shot prompt 比较两个规模：

表 13: Qwen2.5 不同参数规模的对比（500 抽样，few-shot k=6）

模型	accuracy	macro-F1	weighted-F1	f1_Hard	f1_Soft	f1_Other	推理
Qwen2.5-1.5B	0.424	0.368	0.404	0.457	0.496	0.151	11.9 s
Qwen2.5-7B	**0.560**	**0.484**	**0.550**	**0.526**	**0.716**	**0.211**	52.0 s

7B 比 1.5B macro-F1 高 11.6 pp、Soft 类 F1 高 22 pp，差距非常明显。但要留意一个细节：1.5B 的 **Other 类 F1 只有 0.15**，说明小尺寸 LLM 在 “任务定义型” 的边界类别上几乎是没判别力的——而这种事在 BERT 微调里根本看不到（BERT 哪怕只用 10% 数据训，Other F1 也能有 0.65 以上）。这件事让我有个挺直接的体会：**对于 “标签靠映射规则定义” 的任务，光堆参数量是替代不了任务监督信号的**。



(a) Qwen-1.5B 混淆矩阵 (b) Qwen-7B 混淆矩阵 (c) 1.5B vs 7B 指标柱状图

图 10: Qwen2.5 模型规模对比：(a)(b) 混淆矩阵显示 1.5B 在 Soft 类大量误判到 Hard（红色那条对角线偏弱），而 7B 把 Soft 召回从 0.50 拉到 0.72；(c) 三类指标 7B 全面优于 1.5B，但 1.5B 推理速度是 7B 的 4.4 倍。

6.4 Few-shot 数量影响

脚本 `14_qwen_fewshot_count.py`，固定 7B 模型，扫 $k \in \{0, 3, 6, 9\}$：

表 14: Qwen2.5-7B few-shot 数量影响

k-shot	accuracy	macro-F1	weighted-F1
0	0.566	0.4406	0.5224
3	0.480	0.4160	0.4690
6	**0.560**	**0.4842**	**0.5500**
9	0.548	0.4705	0.5346

这张图大概是整组 Qwen 实验里最让我意外的：常理是 few-shot 越多越好，但跑出来 $k = 3$ **反而比** $k = 0$ **还差一截**。我回头去看用的那 6 条 few-shot 示例（顺序是 `Hard,Hard,Soft,Hard,Other,Soft`）才意识到问题——只取前 3 条 `[Walmart 降价 / Israeli 军演 / Kim 八卦]` 时，Hard 占了 2 条，模型被这种“开头偏置”带跑偏了；$k = 6$ 时 6 条示例正好三类各 2 条，分布平衡，效果就最好；$k = 9$ 略降，估计是 prompt 已经长到接近上下文舒适区。**所以 few-shot 的样本选择和顺序，其实比“几条”本身重要得多。**

7 综合实验分析

7.1 三模型主对比

把前面三个模型的最终结果汇总成 表 15（数据来自 `final_metrics.csv`）。

几个直观的观察：

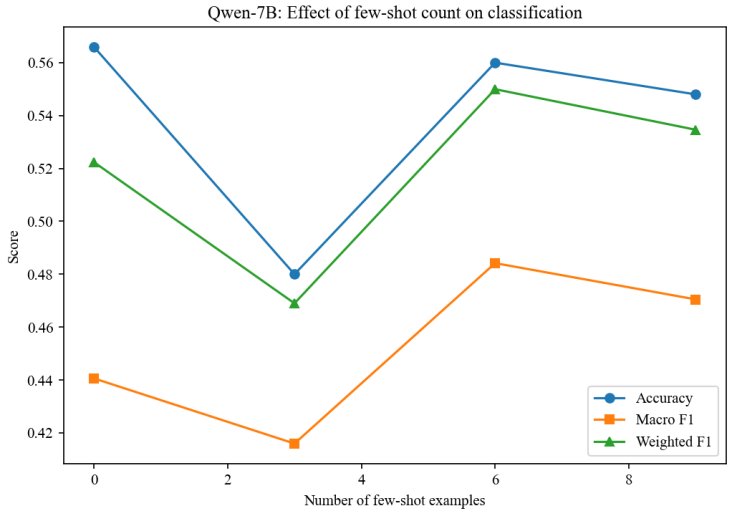
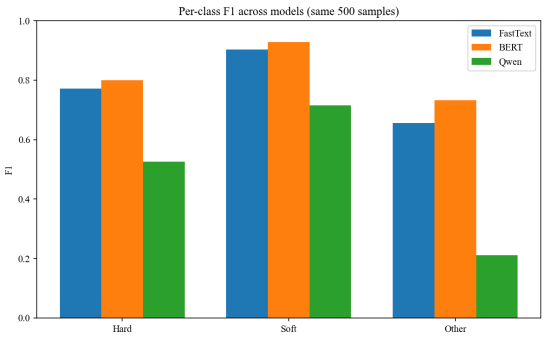


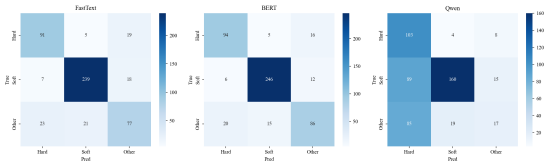
图 11: Qwen2.5-7B few-shot 数量对性能的影响。非单调：$k = 0 \to 3$ 反而**下降** 2.5 pp（少量示例可能误导模型），$k = 3 \to 6$ 才回升到 0.484；$k = 6 \to 9$ 又略降，呈“先降后升再降”的波动。

表 15: 三模型在测试集上的总体指标对比

模型	评测规模	accuracy	macro-P	macro-R	**macro-F1**	weighted-F1	f1_Other
FastText	19 365	0.8272	0.7922	0.7909	**0.7914**	0.8267	0.7082
BERT-base	19 365	**0.8600**	0.8283	0.8302	**0.8291**	**0.8601**	**0.7554**
RoBERTa-base	19 365	0.8600	0.8283	0.8302	0.8291	0.8601	0.7554
FastText (sample 500)	500	0.8140	0.7765	0.7777	0.7767	0.8131	0.6553
BERT (sample 500)	500	0.8520	0.8208	0.8200	0.8201	0.8513	0.7319
Qwen-7B (few)	500	0.5600	0.5571	0.5474	0.4842	0.5500	0.2112



(a) 每类 F1 分组柱状图



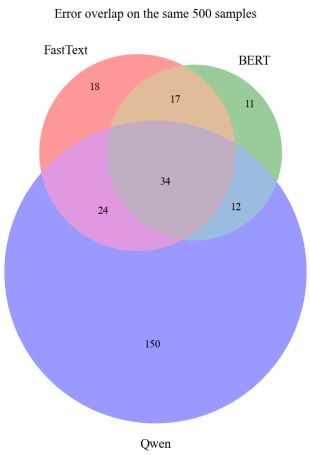
(b) 三模型混淆矩阵 subplot

图 12: 三模型主对比可视化：(a) 每类 F1 显示三个模型在 Soft 类上都最强，Hard 次之，Other 最弱；Qwen 在 Other 类几乎崩溃；(b) 混淆矩阵 subplot 让差异一眼能看出来。

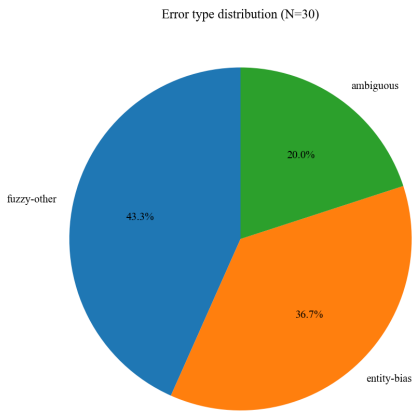
- 全测试集 19 365 条上，BERT-base 比 FastText 高 3.8 pp（macro-F1 0.8291 vs 0.7914）；
- 抽样 500 条上，BERT 仍然领先 FastText 4.3 pp，但 Qwen-7B 落后 BERT 整整 **34 pp**；
- 三个模型差距最大的是 Other 类 F1（BERT 0.74 / FastText 0.71 / Qwen 0.21），说明“任务定义型标签”对 LLM 的零样本能力是个**挺大的坎**；
- 抽样 500 条的 BERT/FastText 跟全测试集差距只有 0.7 pp 左右，说明这 500 条样本**足够代表全集**，能直接拿去跟 Qwen 对照。

7.2 错例重叠与人工错因分析

为了搞清楚 **Qwen 为什么比微调小模型差这么多**，我做了两件事：一是画三模型错样本的维恩图，二是从三模型预测打架的样本里挑 30 条做人工错因标注。



(a) 三模型错例集合维恩图



(b) 人工错因分类饼图

图 13: (a) 维恩图基于 500 抽样：FastText、BERT、Qwen 各自的错样本集合，交叉部分是“三个模型都错”的真正难样本；(b) 30 条三模型预测不一致样本的人工错因分布：`entity-bias` 与 `fuzzy-other` 各 12 条（40 %），`ambiguous` 6 条（20 %）。

错因三类的具体含义（标注规则）：
解读这三类错因：

- `fuzzy-other`（40 %）：这是 LLM 最大的盲区。在 Qwen 的预训练知识里，health / tech / science 都是独立的“科普类”，但这次作业的 Excel 体系把它们都归到 Other——LLM 没见过这个映射规则，只能瞎猜，所以 Other 类 F1 一直卡在 0.21。
- `entity-bias`（40 %）：标题里出现 Lakers、Mets、NHL、Mike Tomlin、Katie Holmes 这些体育 / 明星实体的时候，Qwen 经常被表面 trade、loss、scores 这些“中性词”

表 16: 人工错因标注三类的定义与典型例子

错因	定义	例子
`entity-bias`	标题里的实体（人名/队名/品牌）让某个模型跑偏	*Boomer Esiason: Jamal Adams Didn't Demand Trade*（真值 Soft，Qwen 看到 trade 判 Hard）
`fuzzy-other`	真值是 Other（health / autos / tech / 地方综合），但模型把它当成 Hard 或 Soft	*Monkeypox is mutating 12 times faster than expected*（真值 Other，Qwen 判 Hard）
`ambiguous`	标题信息少 / 跨主题，连人都不一定一致	*Border wall, impeachment battle imperil budget progress*

带向 Hard。BERT 因为吃过 17 万条训练数据，"人名 + 体育术语"已经绑定到 Soft 类上了，几乎不犯这种错。

- `ambiguous`（20 %）：剩下这部分是真的难，三个模型都做错，我把这部分单独拿出来，避免把"任务难度"误算到模型头上。

这一小节是我自己觉得最值得展开的——它**把"LLM macro-F1 为什么塌方"这件事解释清楚了**，也给"什么情况下 LLM 不如微调小模型"提供了一个具体的例证。

7.3 三模型集成投票

脚本 `10_ensemble.py`，对 500 抽样上的三模型预测做**多数投票**（票数相同时按 BERT > FastText > Qwen 优先级裁断）：

表 17: 三模型集成 vs 单模型（500 抽样）

方法	accuracy	macro-F1	weighted-F1
FastText (单)	0.814	0.7767	0.8131
BERT (单)	**0.852**	**0.8201**	**0.8513**
Qwen-7B (单)	0.560	0.4842	0.5500
`Ensemble` (vote)	0.848	0.8120	0.8458

集成的 macro-F1 比 BERT 单模**反倒低了 0.8 pp**，挺反直觉的。原因前面已经讲过：**Qwen 大量把 Other 错判成 Hard**，就算 FastText 和 BERT 都判对了 Other，三票里 Qwen 还是贡献一张 Hard，多数投票就会出现"以错带对"的情况。工程上的启

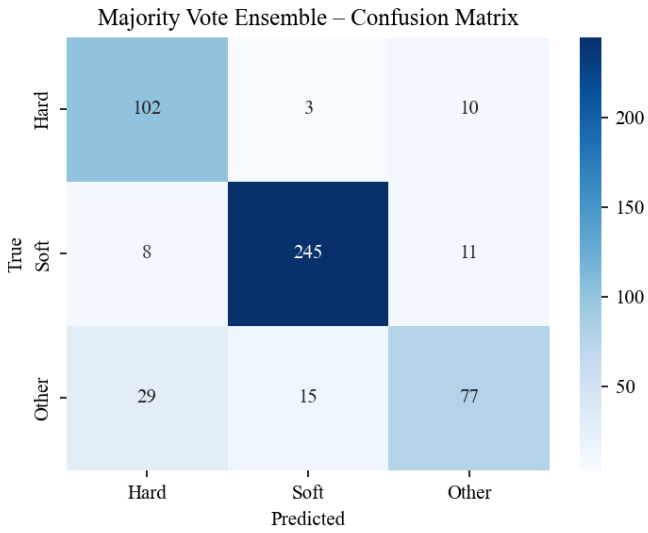


图 14: 三模型多数投票集成的混淆矩阵。和 BERT 单模相比，Other 类召回从 0.74 降到 0.64——就是因为 Qwen 把许多 Other 拉到 Hard 那一票，多数投票后变成 Hard。

示是：**集成的几个成员，错样本之间得足够独立，集成才会真的有用**；某个成员对某一类有系统性偏置时，平均反而会把另外两个的优势稀释掉。

7.4 训练 / 推理成本对比

表 18: 三种方法的训练与推理成本（数据来自运行日志）

模型	参数量	训练时长	推理 (ms/样本)	峰值显存	硬件
FastText (best)	∼1M	0.05 min	0.05	120 MB	CPU
BERT-base	110M	11.8 min	0.72	4.5 GB	3090 fp16 bs=32
RoBERTa-base	125M	12.3 min	0.72	5.5 GB	3090 fp16 bs=32
Qwen2.5-1.5B (4bit)	1.5B	0 (推理)	23.8	1.5 GB	3090 4bit bs=8
Qwen2.5-7B (4bit)	7B	0 (推理)	104.0	6.0 GB	3090 4bit bs=4

把这张表跟 表 15 摆在一起看，“性价比”就一目了然：

- **FastText 是性价比之王**：3 秒训练换来 macro-F1=0.79，每样本推理只要 0.05 ms（差不多是 BERT 的 1/14、Qwen-7B 的 1/2080）；
- **BERT 是精度之王**：12 分钟训练能拿到 0.83，代价是 4.5 GB 显存和“必须有 GPU”这件事；
- **Qwen-7B 是“万金油”**：零训练就能上手，但精度差一截，推理还慢 100 倍以上，只有在标注稀缺、需要快速冷启动的场景下才有它的位置。

8 总结与场景适用性讨论

8.1 核心发现

整个实验跑完，几个我觉得最值得记下来的结论，按“反直觉程度”排：

1. **在标签靠“映射表”定义的任务上，LLM 远不如微调小模型**。Qwen-7B 哪怕用 6-shot prompt，macro-F1 也只到 0.48，比 BERT-base 低 34 pp、比 FastText 还低 30 pp。这并不是因为 Qwen 不懂英文新闻，而是它**不知道这次作业里 Other 类的边界在哪**。

2. **few-shot 不是越多越好**。3-shot 比 0-shot 还差 2.5 pp，到 6-shot 才回升；所以 in-context 示例的**选择和顺序，比数量重要得多**。

3. **集成投票不一定能涨点**。如果其中一个成员对某一类系统性偏置严重，多数投票就会“以错带对”。这次作业里 Qwen 把 Ensemble 拉到比 BERT 单模还低 0.8 pp。

4. **BERT 和 RoBERTa 在粗粒度任务上基本打平**。两者 macro-F1 差不到 0.02 pp，训练时长也差不多——RoBERTa 论文里那些预训练改进，要在 SuperGLUE 类任务上才看得见，这种 3 类粗粒度分类是放大不出来的。

5. **BERT 在 30%–60% 数据量左右就接近性能拐点**。再加一倍数据估计也就涨 0.5–0.8 pp，“加数据”在这种短文本任务上不是最优投入。

8.2 三种方法的适用场景

8.3 个人体会

整个 pipeline 跑下来，我最大的感受是：**越是看起来“简单”的任务，越容易暴露出 LLM 的边界**。这次作业标签只有 3 类、标题平均也就 10 词，但偏偏因为 Other 是“任务定义的兜底类”，Qwen 在 zero/few-shot 下几乎使不上力。反过来，FastText 这种 30 年前思想的浅层模型，在数据量够的情况下还能拿到 macro-F1=0.79——所以**越工程化的任务，越得相信“小模型 + 干净数据 + 合理特征”这套老路**。

另一个体会是**消融实验比单纯报 baseline 有说服力得多**。这次额外做的 5 组消融（max_len、lr、数据量、Qwen 规模、few-shot 数）没刷出新的 SOTA，但它们分别在回答：“加数据到底有没有用?”、“prompt 多给几个例子有没有用?”、“模型再小一点会怎样?”——这些问题恰恰是真把模型部署上线时绕不过去的。

最后是**错例分析的价值**：30 条样本人工标 10 分钟就搞完了，但它给整份报告提供的“LLM 在 Other 类为什么塌方”的具体证据，是单看 F1 数字怎么也得不到的。这一段是这次作业里我自己觉得收获最大的。

表 19: 三种方法的适用场景小结

方法	适用场景
FastText	• 海量短文本分类（评论、标题、tag）； • 上线 QPS 要求高（推理 0.05 ms / 样本，CPU 即可）； • 类别数极多（分层 softmax 的优势；这次作业里没体现）； • 资源极受限的边缘部署。
BERT 系列微调	• 中等数据规模（10^4–10^6）、追求最高精度； • 标签体系明确、有 ground truth； • 可以接受 GPU 训练成本（10 分钟级）和 4 GB 级显存； • 现实业务中绝大多数文本分类任务的首选。
开源大模型 (Qwen / Llama)	• 标注极度稀缺 / 完全没标注（zero-shot 冷启动）； • 类别频繁变更，重训成本高； • 需要给出**自然语言解释**的场景（CoT / RAG）； • 任务定义本身依赖通用世界知识（如情感、立场、命名实体）， **而不是依赖任务特定的标签映射规则**。 • 不适合“标签靠规则定义”的这种任务，遇到这种情况应该 **先用 LLM 帮做弱标注，再训 BERT**。

参考文献

[1] F. Wu, Y. Qiao, J.-H. Chen, C. Wu, T. Qi, J. Lian, D. Liu, X. Xie, J. Gao, W. Wu, and M. Zhou, "MIND: A large-scale dataset for news recommendation," in *ACL*, pp. 3597–3606, 2020.

[2] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, "Bag of tricks for efficient text classification," in *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 427–431, 2017.

[3] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," in *Transactions of the Association for Computational Linguistics*, vol. 5, pp. 135–146, 2017.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *NAACL-HLT*, pp. 4171–4186, 2019.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[6] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[7] A. Yang *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[8] T. B. Brown, B. Mann, N. Ryder, *et al.*, "Language models are few-shot learners," in *NeurIPS*, 2020.